VISIONX

MARKETING AUTOMATION

TÂCHE 1 — ANALYSE DES EXIGENCES ET CONCEPTION DE L'ARCHITECTURE TECHNIQUE

Document de travail de l'équipe VisionX
Phase de démarrage • Livrable préparatoire L1

Élément	Description
Projet	Agent logiciel de Marketing Automation
Équipe	VisionX — 6 membres
Objectif du document	Définir la démarche d'analyse, les fonctionnalités, l'architecture proposée et les travaux à réaliser avant le développement.
Statut	Proposition de conception à valider avec les contrats officiels de l'enseignant

Version préparatoire — 7 novembre 2026

Sommaire

- 1. Compréhension du projet et périmètre fonctionnel
- 2. Analyse des documents contractuels
- 3. Architecture fonctionnelle générale
- 4. Architecture technique proposée
- 5. Communication avec le noyau central
- 6. Conception préliminaire de la base de données
- 7. Scénarios métier marketing
- 8. Exigences non fonctionnelles et sécurité
- 9. Répartition des responsabilités
- 10. Planning de réalisation
- 11. Checklist de validation
- 12. Livrable L1 et prochaines actions

Important — statut des informations

Ce document formalise une proposition de travail construite à partir des informations déjà évoquées dans la conversation : agent Marketing Automation, objets métier Prospect (produit) et Commande (consommé), et technologies proposées Mautic, Postfix et Python. Les trois documents contractuels complets n'ont pas encore été examinés ici. Les endpoints, schémas JSON, règles d'authentification et protocoles exacts restent donc à confirmer. Les exemples techniques de ce document sont illustratifs et ne constituent pas des formats contractuels.

1. Compréhension du projet et périmètre fonctionnel

1.1. Présentation

L'équipe VisionX travaille sur un agent logiciel spécialisé dans l'automatisation marketing. Sa mission proposée est d'aider une entreprise à gérer des prospects, préparer et exécuter des campagnes, automatiser certaines communications et suivre les résultats. L'agent doit fonctionner comme un composant autonome tout en s'intégrant à la plateforme centrale de l'enseignant.

1.2. Objets métier connus

Nature	Objet	Rôle de l'agent
Produit / publié	Prospect	Notre agent doit pouvoir publier un prospect dans le noyau central, selon le contrat.
Consommé / reçu	Commande	Notre agent doit pouvoir recevoir et traiter une commande provenant du noyau central, selon le contrat.

1.3. Fonctionnalités envisagées

- Gestion des prospects : création, consultation, qualification et suivi des prospects.
- Gestion des campagnes : création, configuration, planification et suivi des campagnes.
- Automatisation email : préparation et envoi de messages selon des règles autorisées.
- Segmentation : regroupement de prospects selon des critères définis.
- Déclencheurs : exécution d'actions à la réception d'événements ou selon des conditions.
- Statistiques : suivi des envois, erreurs et résultats disponibles.

Cette liste constitue un périmètre fonctionnel initial. Elle doit être comparée au cahier des charges et aux exigences contractuelles avant d'être considérée comme définitive.

2. Analyse des documents contractuels

L'analyse des contrats est la première activité obligatoire avant de figer l'architecture ou de coder. Chaque règle doit être relevée, comprise, associée à une source et transformée en exigence vérifiable.

2.1. AGENT_INTEGRATION_CONTRACT.doc

Ce document définit les règles de connexion de notre agent à la plateforme centrale. Les endpoints cités ci-dessous ont été mentionnés dans les éléments déjà partagés ; leur présence, leur méthode HTTP, leurs paramètres et leurs réponses doivent être vérifiés dans le fichier original.

Endpoint mentionné	Rôle attendu
GET /health	Vérifier l'état de fonctionnement de l'agent.
POST /tenant/activate	Activer l'agent pour un tenant.
POST /tenant/deactivate	Désactiver l'agent pour un tenant.
GET /tenant/{id}/status	Consulter l'état de l'agent pour un tenant.
GET /config-schema	Exposer le schéma de configuration.

Points à extraire du contrat :

- Méthodes HTTP, chemins, paramètres, corps des requêtes et codes de réponse exacts.
- Format standard des réponses et valeurs possibles du champ status.
- Format normalisé des erreurs et règles de validation.
- Mécanisme d'authentification, autorisations et gestion des secrets.
- Règles d'activation/désactivation et comportement en cas d'échec.
- Exigences de logs, disponibilité, configuration et isolation des tenants.

Une enveloppe de réponse illustrative évoquée précédemment est :

```
{ "status": "success", "data": {}, "error": null }
```

Ne pas supposer que cet exemple est exhaustif : le contrat complet fait foi.

2.2. INTER_AGENT_DATA_CONTRACT.doc

Ce document définit les objets échangés entre agents. Pour notre agent, les informations déjà identifiées sont Prospect comme objet produit et Commande comme objet consommé.

- Relever le nom exact de chaque objet et sa version éventuelle.
- Lister les champs obligatoires et facultatifs, leurs types et contraintes.
- Identifier les identifiants, dates, relations et règles de validation.
- Comprendre comment un objet est publié, distribué, reçu et acquitté.
- Identifier les événements, abonnements ou mécanismes de notification.
- Vérifier les règles de compatibilité et d'évolution des schémas.

Ne pas inventer un schéma Prospect ou Commande. Les exemples de payload doivent être copiés ou dérivés du contrat officiel uniquement.

2.3. DEVELOPER_INSTRUCTIONS.doc

- Instructions de lancement local et prérequis logiciels.
- Structure attendue du dépôt et conventions de développement.

- Exigences Docker et modalités de remise.
- Tests, commandes de validation et critères d’acceptation.
- Format des livrables, documentation et éventuelle démonstration.

2.4. Tableau de conformité à compléter

Exigence	Source	Statut initial
Endpoints obligatoires	Integration Contract	À vérifier dans le contrat
Format des réponses et erreurs	Integration Contract	À vérifier
Authentification et isolation tenant	Integration Contract	À vérifier
Publication de Prospect	Data Contract	À vérifier
Consommation de Commande	Data Contract	À vérifier
Notifications / événements	Data Contract	À vérifier
Packaging et lancement	Developer Instructions	À vérifier
Tests et remise	Developer Instructions	À vérifier

3. Architecture fonctionnelle générale

3.1. Vue d’ensemble

L’architecture fonctionnelle proposée sépare l’interface, le backend métier, les services marketing, le stockage et le module d’intégration. Le noyau central reste l’intermédiaire des échanges inter-agents conformément aux règles de la plateforme.



Le schéma représente les responsabilités logiques. Les flux précis (HTTP, webhooks, polling, messages ou autre) doivent être déterminés à partir des contrats.

3.2. Description des composants

Composant	Responsabilités
Frontend	Interface pour consulter les prospects, configurer les campagnes, gérer les règles et consulter les statistiques.
Backend	API, logique métier, validation, permissions, orchestration et communication avec les services.
Base de données	Stockage des prospects, campagnes, templates, règles, journaux et paramètres par tenant.
Service email	Préparation et transport des messages, suivi des erreurs et résultats d’envoi.
Moteur d’automatisation	Évaluation des déclencheurs et exécution des actions métier autorisées.
Module d’intégration	Adaptation aux contrats, réception/publication des objets et gestion des erreurs d’échange.

3.3. Flux fonctionnel simplifié

Événement reçu → validation du tenant et du schéma → application des règles métier → action marketing autorisée → journalisation du résultat. Pour une publication : données validées → transformation conforme au contrat → transmission au noyau → traitement de la réponse.

4. Architecture technique proposée

4.1. Stack technique initiale

Couche	Technologie envisagée	Raison / rôle
Frontend	React + Vite	Interface web réactive.
UI	Tailwind CSS	Mise en forme rapide et cohérente.
Backend	Python + Django REST Framework	API, permissions, validation et logique métier.
Base de données	PostgreSQL	Stockage relationnel et contraintes.
Email	Postfix / SMTP	Transport des emails.
Marketing	Mautic	Plateforme marketing possible, à évaluer.
Packaging	Docker	Environnement reproductible et déploiement.
Versioning	Git + GitHub	Collaboration et historique du code.
Tests	Pytest et outils DRF	Tests unitaires et d'intégration.

4.2. Décision à prendre concernant Mautic

Le fait que Mautic, Postfix et Python soient cités comme outils proposés ne signifie pas automatiquement qu'ils sont tous obligatoires. Une décision doit être prise après lecture des consignes. Une approche possible consiste à construire le backend métier Python et à intégrer Mautic uniquement si ses fonctions apportent une valeur claire et ne doublonnent pas les modules développés.

Postfix est un agent de transfert de courrier (MTA) : il assure le transport SMTP. Il ne remplace pas à lui seul un outil de gestion de campagnes, de segmentation ou d'automatisation.

4.3. Organisation proposée du backend

```
backend/
■■■ config/ (settings.py, urls.py, wsgi.py)
■■■ apps/
■ ■■■ tenants/
■ ■■■ prospects/
■ ■■■ campaigns/
■ ■■■ automation/
■ ■■■ email_service/
■ ■■■ analytics/
■ ■■■ integration/
■■■ common/ (exceptions.py, permissions.py, responses.py)
■■■ tests/
■■■ manage.py
■■■ requirements.txt
■■■ Dockerfile
```

Chaque application doit conserver une responsabilité claire. Le module integration ne doit pas disperser les règles du contrat dans tous les modules métier.

5. Communication avec le noyau central

5.1. Communication entrante : Commande

Scénario logique : un agent producteur publie une commande ; le noyau central l'enregistre et la distribue selon ses mécanismes ; notre agent reçoit l'objet ; il valide le tenant et le schéma ; le moteur évalue les règles marketing ; une action éventuelle est exécutée et journalisée.

Pseudo-code illustratif (le format réel de order doit venir du contrat) :

```
def process_order(order):
if not order:
return
# Valider le schéma contractuel
# Vérifier le tenant et les permissions
# Détecter les doublons d’événements
# Évaluer les règles marketing actives
# Exécuter une action autorisée
# Enregistrer le résultat
```

5.2. Communication sortante : Prospect

- Un prospect est créé ou qualifié dans notre agent.
- Le backend valide les données et vérifie le tenant.
- Le module d’intégration construit l’objet selon le schéma officiel.
- L’agent publie l’objet via le mécanisme prescrit par le noyau.
- La réponse du noyau est contrôlée et le résultat est journalisé.

5.3. Principes d’intégration

- Ne pas connecter directement deux agents si le noyau impose un passage central.
- Respecter les noms, versions et schémas officiels des objets métier.
- Gérer les erreurs réseau, les réponses invalides et les reprises.
- Prévoir l’idempotence pour éviter de traiter deux fois le même événement.
- Ne jamais publier de données personnelles sans base légale/consentement approprié.
- Tracer les échanges sans écrire de secrets ou de données sensibles inutiles dans les logs.

6. Conception préliminaire de la base de données

6.1. Entités proposées

Entité	Champs indicatifs	Rôle
Tenant	id, name, status	Espace d’une entreprise.
Prospect	id, tenant_id, email, name, status	Données de prospects propres au tenant.
Campaign	id, tenant_id, name, status	Campagnes marketing.
EmailTemplate	id, tenant_id, subject, content	Modèles de messages.
AutomationRule	id, tenant_id, trigger, action, enabled	Règles de déclenchement.
EmailLog	id, tenant_id, status, sent_at	Historique des envois.
IntegrationEvent	id, tenant_id, event_type, status	Suivi des événements échangés.

Ces champs sont indicatifs et non contractuels. Le schéma final doit tenir compte des exigences métier, des objets partagés et des données réellement nécessaires.

6.2. Relations à étudier

- Un tenant possède plusieurs prospects, campagnes et règles d’automatisation.
- Une campagne peut cibler plusieurs prospects : une relation plusieurs-à-plusieurs peut être nécessaire.
- Une campagne peut utiliser un ou plusieurs templates selon le besoin retenu.
- Une règle d’automatisation peut déclencher une action ou une campagne.
- Un email envoyé doit être traçable dans un journal.
- Un événement d’intégration doit conserver son état de traitement et un identifiant de déduplication.

6.3. Isolation multi-tenant

Toutes les données propres à une entreprise doivent être protégées contre l’accès d’un autre tenant. Les contrôles doivent être réalisés côté serveur, pas uniquement dans l’interface.

- Associer les enregistrements métier au tenant lorsque cela est pertinent.
- Déduire le tenant à partir d’un contexte authentifié et vérifié, jamais d’une valeur client non fiable seule.
- Filtrer toutes les lectures et écritures par tenant.
- Contrôler les permissions sur chaque endpoint.
- Tester explicitement qu’un tenant ne peut ni lire ni modifier les données d’un autre tenant.

7. Scénarios métier marketing

7.1. Campagne de bienvenue

- Un prospect est ajouté au système.
- Le système vérifie la validité de son adresse et son consentement marketing.
- Le moteur recherche une règle de bienvenue active.
- Le template adapté est sélectionné.
- Le message est transmis au service SMTP si les conditions sont réunies.
- Le résultat est enregistré : accepté, envoyé, échoué ou autre statut défini.
- Les statistiques sont actualisées.

7.2. Campagne liée à une commande

- Une commande est reçue depuis le noyau central.
- L'événement et le tenant sont validés.
- Le système vérifie si l'événement a déjà été traité.
- Les règles marketing applicables sont évaluées.
- Le consentement, les préférences et les règles métier sont contrôlés.
- Une action autorisée peut être déclenchée.
- Le résultat est journalisé.

Important : une commande ne doit pas déclencher automatiquement un email dans tous les cas. Le déclenchement dépend des règles validées, des préférences du destinataire et des contraintes légales.

8. Exigences non fonctionnelles et sécurité

Domaine	Exigences / mesures à prévoir
Sécurité	Authentification, autorisations, secrets protégés, validation des entrées.
Multi-tenant	Isolation stricte des données et tests inter-tenants.
Fiabilité	Gestion des erreurs, reprise contrôlée, idempotence et files d'attente si nécessaire.
Performance	Traitement asynchrone des tâches longues et pagination des listes.
Traçabilité	Logs structurés, identifiants de corrélation et historique des événements.
Confidentialité	Minimisation des données, consentement, désabonnement et conservation maîtrisée.
Maintenabilité	Modules séparés, documentation et conventions de code.
Portabilité	Image Docker reproductible et configuration par variables d'environnement.
Qualité	Tests unitaires, API, intégration et scénarios d'erreur.

8.1. Risques initiaux

Risque	Conséquence	Mesure de réduction
Mauvaise interprétation du contrat	Agent non intégrable	Lire les contrats et construire une matrice de conformité.
Schéma métier incorrect	Rejet des objets	Utiliser les schémas officiels et ajouter des tests de contrat.
Fuite entre tenants	Incident de confidentialité	Permissions côté serveur et tests d'isolation.
Événement reçu plusieurs fois	Actions ou emails dupliqués	Idempotence et identifiant d'événement.
Échec SMTP	Email non transmis	Journalisation, reprise contrôlée et suivi des erreurs.
Empilement d'outils	Complexité et doublons	Valider la nécessité de Mautic et des autres composants.

9. Répartition des responsabilités dans l'équipe

Répartition proposée pour les six membres de VisionX. Elle peut être ajustée selon les compétences et la disponibilité de chacun.

Membre	Responsabilité principale	Résultat attendu
Bouharizi Mohamed Anes	Architecture globale et coordination	Diagramme général et synthèse des décisions.
Sebti Iyed	Contrat d'intégration	Tableau des endpoints, erreurs et exigences d'intégration.
Zouane Akram	Contrat des données inter-agents	Fiches Prospect et Commande avec champs exacts.
Fellah Baha	Modélisation des données	MCD/MLD et dictionnaire de données préliminaire.
Bertil Anis	Analyse métier marketing	Cas d'utilisation, scénarios et règles fonctionnelles.
Bahi Adem	Étude technique et sécurité	Comparatif de stack, risques et exigences de sécurité.

9.1. Règle de collaboration

- Chaque responsable documente ses décisions et cite le passage du contrat qui les justifie.
- Les questions non résolues sont inscrites dans un registre de questions.
- Les décisions d'architecture sont validées collectivement.
- Les changements de schéma ou d'interface sont communiqués à toute l'équipe.
- Le code ne commence qu'après validation des exigences indispensables.

10. Planning de réalisation de la tâche 1

Période	Activités	Résultat
Jour 1	Réunion de démarrage ; lecture des trois contrats ; extraction des exigences.	Cahier des exigences et questions ouvertes.
Jour 2	Définition des fonctions ; architecture fonctionnelle et technique ; flux inter-agents.	Diagrammes et choix techniques provisoires.
Jour 3	Modèle de données ; sécurité ; risques ; revue collective.	Architecture validée et préparation du livrable L1.

Ce planning est indicatif : il doit être adapté à la date de remise et à la disponibilité des documents contractuels.

11. Checklist de validation

N°	Élément à valider	État
1	Lire intégralement les trois documents contractuels.	☐ À faire
2	Identifier les endpoints obligatoires et leurs formats exacts.	☐ À faire
3	Documenter l'authentification, les erreurs et les règles tenant.	☐ À faire
4	Extraire le schéma officiel de Prospect.	☐ À faire
5	Extraire le schéma officiel de Commande.	☐ À faire
6	Comprendre les mécanismes de publication et de réception.	☐ À faire
7	Définir et prioriser les fonctionnalités de la première version.	☐ À faire
8	Produire le diagramme d'architecture fonctionnelle.	☐ À faire
9	Produire le diagramme des composants techniques.	☐ À faire
10	Décrire les séquences de réception et publication.	☐ À faire
11	Concevoir le modèle de données préliminaire.	☐ À faire
12	Définir les contrôles multi-tenant et de confidentialité.	☐ À faire
13	Valider la stack technique et la place éventuelle de Mautic.	☐ À faire
14	Documenter les risques et les décisions ouvertes.	☐ À faire
15	Préparer et relire le livrable L1.	☐ À faire

12. Livrable L1 et prochaines actions

12.1. Contenu attendu du L1

D'après les éléments du cahier des charges déjà évoqués, le L1 est un document d'orientation court (environ une page Word) à remettre avant T2. Il doit présenter les éléments suivants :

- Agent choisi et domaine.
- Objectif de l'agent et justification du choix.
- Méthodologie et plan prévisionnel.
- Sources et documents de référence.
- Contrat d'intégration pris en compte.
- Registre des objets métier partagés.
- Outils open source envisagés.

- Principaux risques identifiés.

Le présent document est un dossier de travail détaillé pour réaliser la tâche 1 ; il ne remplace pas nécessairement le format officiel L1 demandé par l’enseignant.

12.2. Prochaines actions immédiates

- Récupérer les trois fichiers : AGENT_INTEGRATION_CONTRACT.doc, INTER_AGENT_DATA_CONTRACT.doc et DEVELOPER_INSTRUCTIONS.doc.
- Attribuer les documents aux responsables indiqués dans la répartition.
- Construire la matrice de conformité avec les références précises aux clauses.
- Organiser une réunion de validation pour résoudre les ambiguïtés.
- Figer l’architecture uniquement après validation des exigences contractuelles.

> Recommandation de l’équipe
> Ne pas commencer le développement avant d’avoir vérifié les contrats d’intégration et de données. Une application marketing fonctionnelle mais incompatible avec le noyau central ne satisfera pas l’objectif d’intégration du projet.

Fin du document — VisionX / Marketing Automation